



## Problem Statement 1:

### Cloud FinOps Intelligence: AI-Powered Multi-Cloud Cost Anomaly Detection and Spend Forecasting

#### Description:

Organizations operating across multiple cloud providers (AWS, Azure, GCP) face an escalating challenge: cloud spend is growing 20-30% year-over-year, yet finance and engineering teams lack the predictive intelligence to detect cost anomalies before they snowball into budget overruns. Traditional FinOps dashboards provide retrospective visibility, but they fail to identify subtle spending deviations, seasonal cost patterns, or forecast future cloud bills with the granularity needed for proactive budget management.

Your challenge is to build an **AI-driven FinOps Anomaly Detection and Forecasting platform** that ingests multi-cloud billing data (AWS Cost Explorer, Azure Cost Management, GCP Billing Export), identifies spending anomalies in real-time using advanced time-series techniques, and delivers accurate multi-horizon spend forecasts to enable proactive financial planning.

#### Challenge Requirements:

- 1. Multi-Cloud Data Ingestion and Normalization:** Build connectors to ingest billing data from at least two major cloud providers (AWS CUR files, Azure Cost Management API, GCP BigQuery Billing Export). Normalize the data into a unified cost taxonomy covering compute, storage, networking, and managed services. Handle data at hourly/daily granularity and ensure the pipeline can process at least 6 months of historical data for model training.
- 2. Time-Series Anomaly Detection Pipeline:** Implement and compare at least three anomaly detection approaches: (a) Statistical methods (Z-score with seasonal decomposition, GESD), (b) ML-based (Isolation Forest, One-Class SVM on engineered cost features), and (c) Deep learning (LSTM Autoencoder or Transformer-based anomaly detection). The system must detect sudden cost spikes, gradual drift (slow creep in spending), seasonal deviations (costs that are normal for one period but anomalous for another), and cross-service correlated anomalies (e.g., compute and network costs spiking together).
- 3. Anomaly Root Cause Attribution:** When an anomaly is detected, automatically trace it back to specific resources, services, teams, or cost allocation tags. Present a causal attribution chain that explains why the anomaly occurred (e.g., an auto-scaling group triggered by a failed health check loop, or a data transfer spike caused by a misconfigured backup job). Rank contributing factors by their cost impact using feature importance or SHAP values.
- 4. Multi-Horizon Spend Forecasting:** Build a forecasting engine that produces accurate cloud spend predictions at 7-day, 30-day, and 90-day horizons, broken down by service, team, and environment. Implement at least two forecasting models (e.g., Prophet and LightGBM with time-series features) and an ensemble that selects the best model per cost segment based on backtesting performance. Provide probabilistic forecasts with confidence intervals (10th, 50th, 90th percentiles) to support budget planning under uncertainty.

## **Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)



**5. Alerting Dashboard and Budget Breach Prediction:** Build a real-time dashboard that surfaces detected anomalies with severity scores, shows forecasted spend trajectories overlaid with budget thresholds, and predicts the probability and estimated date of budget breach for each team/service. Implement configurable alert rules that notify stakeholders when anomaly severity exceeds thresholds or when forecasted spend is projected to exceed budget within the forecast horizon.

### **Suggested Datasets:**

AWS CUR (Cost and Usage Report) sample datasets, Azure Cost Management export CSVs, GCP BigQuery public billing datasets. Participants may also generate synthetic multi-cloud billing data with injected anomalies using provided Python scripts.

### **Technology Requirements:**

Python (pandas, scikit-learn, Prophet/statsmodels, PyTorch for deep learning models), a time-series database (InfluxDB or TimescaleDB), REST API backend (FastAPI/Flask), and a dashboard frontend (Streamlit, Grafana, or React).

### **Evaluation Criteria:**

Anomaly detection F1-score on labeled test data, forecast accuracy (MAPE and WMAPE) across multiple horizons, root cause attribution quality (judged by reviewers), budget breach prediction accuracy, alerting latency, and multi-cloud coverage.

### **Domain:**

[FinOps & Cloud Cost Management](#)

## **Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)



### **Problem Statement 2:**

## **Smart Retail Shelf Intelligence: Computer Vision-Driven Inventory Monitoring and Demand Optimization**

### **Description:**

Retail out-of-stock events cost the global industry an estimated \$1 trillion annually in lost sales. Despite heavy investments in ERP and supply chain systems, the last-mile visibility gap remains: retailers often do not know the real-time state of their shelves until a customer complains or a manual audit is performed. Meanwhile, overstocking ties up capital and leads to markdowns and waste, especially for perishable goods. Existing camera infrastructure in stores is vastly underutilized, primarily serving only security purposes.

Your challenge is to build an **intelligent shelf monitoring and inventory optimization system** that leverages computer vision on existing store camera feeds combined with demand forecasting to provide real-time shelf-level visibility, planogram compliance checking, and automated replenishment recommendations.

### **Challenge Requirements:**

- 1. Shelf Image Analysis and Product Detection:** Use computer vision (YOLO, Faster R-CNN, or Vision Transformers) to analyze shelf images and detect: product presence/absence, stock levels (full, low, empty), planogram compliance (correct product placement), and price tag accuracy. The model should handle varying lighting conditions, camera angles, product orientations, and partial occlusions from customers. Implement SKU-level product recognition to distinguish between similar-looking products on the same shelf.
- 2. Automated Planogram Compliance:** Build a planogram comparison engine that takes the intended shelf layout (provided as a structured JSON or image reference) and compares it against the detected shelf state from camera feeds. Identify misplaced products, missing facings, incorrect price tags, and unauthorized products. Generate compliance scores per aisle and per shelf section.
- 3. Demand Forecasting and Replenishment:** Build a time-series forecasting model that predicts product-level demand at each store location using historical POS data, promotional calendars, weather data, and local event schedules. The forecast should drive automated reorder point calculations and generate replenishment orders with suggested quantities and timing to minimize both stockouts and overstock.
- 4. Real-Time Alert System:** Implement an alerting pipeline that notifies store associates within 5 minutes of detecting a stockout or planogram violation. Alerts should be prioritized by revenue impact and include suggested corrective actions. Support multiple notification channels (dashboard push notifications, mobile alerts, and email digests).
- 5. Dashboard and Analytics:** Create a management dashboard showing shelf health scores, stockout frequency heatmaps by aisle and time of day, planogram compliance trends, demand forecast accuracy metrics, and estimated revenue recovered through faster replenishment. Include a visual shelf map that overlays detection results on store floor plans.

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Suggested Datasets:**

Open-source retail shelf image datasets (Grocery Store dataset, SKU-110K), synthetic POS transaction data with seasonal patterns, sample planogram layouts in JSON format, promotional calendar data, and historical weather data for forecast enrichment.

**Technology Requirements:**

Python (PyTorch/TensorFlow for CV models, scikit-learn/Prophet for forecasting), OpenCV for image processing, PostgreSQL or MongoDB for storage, a message queue (Redis or RabbitMQ) for alert delivery, and a visualization layer (Streamlit/Dash/React).

**Evaluation Criteria:**

Object detection mAP on shelf images, SKU recognition accuracy, planogram compliance detection precision, demand forecast accuracy (WMAPE), alert latency, false positive rate for stockout detection, and overall system integration.

**Domain:**

Retail & AI



### Problem Statement 3:

## E-Commerce Dynamic Pricing and Personalization Engine Using Real-Time Behavioral Signals

### Description:

E-commerce platforms generate vast amounts of behavioral data, including clickstreams, search queries, cart additions, wish-list activity, and browsing session patterns, yet most pricing and recommendation systems operate on stale batch-processed data. This lag means pricing decisions miss real-time demand shifts, and product recommendations fail to adapt to a user's evolving intent within a single session.

Your challenge is to build a **real-time dynamic pricing and personalization engine** that processes user behavioral signals as they happen, adjusts product pricing based on demand elasticity and competitive positioning, and delivers hyper-personalized product recommendations, all within sub-second latency.

### Challenge Requirements:

- 1. Real-Time Event Processing:** Build a streaming data pipeline (using Kafka, Apache Flink, or Redis Streams) that ingests user clickstream events (page views, searches, cart actions, purchases) with sub-second latency. Compute real-time features such as session engagement score, category affinity, and purchase intent probability.
- 2. Dynamic Pricing Model:** Implement a demand-responsive pricing algorithm that considers: real-time demand velocity (how fast a product is being viewed/purchased), competitor price benchmarks (scraped or simulated), inventory levels and restocking timelines, user segment willingness-to-pay estimates. The model must enforce business rules (minimum margins, maximum discount caps, price parity constraints).
- 3. Session-Aware Recommendation System:** Build a hybrid recommendation engine combining collaborative filtering with a session-based model (GRU4Rec or Transformers) that adapts recommendations as the user navigates. The system should handle the cold-start problem for new users using contextual signals (device type, referral source, time of day).
- 4. A/B Testing Framework:** Include a built-in experimentation framework that allows teams to A/B test different pricing strategies and recommendation algorithms with statistical rigor. Track conversion rate, average order value, and revenue per session as key metrics.
- 5. Fairness and Transparency:** Ensure the dynamic pricing model does not discriminate based on user demographics. Implement price explanation features that help users understand why they see a particular price (e.g., limited-time offer, demand-based pricing).

### Suggested Datasets:

Synthetic e-commerce clickstream data (10M+ events), product catalog with 5,000+ SKUs across 50 categories, simulated competitor pricing feeds, and user segment profiles. All datasets will be provided as Parquet and JSON files.

### Technology Requirements:

Apache Kafka or Redis Streams for event processing, Python (scikit-learn, PyTorch for recommendations), a feature store (Feast or custom Redis), FastAPI for serving, and a React/Next.js frontend for the storefront demo.

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Evaluation Criteria:**

Recommendation hit rate and NDCG, pricing model revenue uplift vs. baseline, system latency (p99 < 200ms), A/B test statistical validity, and fairness audit results.

**Domain:**

E-Commerce & Machine Learning



## Problem Statement 4:

### Multi-Signal Time-Series Anomaly Detection for Industrial IoT and Infrastructure Monitoring

#### Description:

Modern infrastructure, from data center cooling systems to manufacturing production lines, generates thousands of correlated time-series signals every second. Detecting anomalies in these signals is critical for preventing equipment failures, reducing downtime, and ensuring safety. However, traditional threshold-based monitoring fails to capture complex inter-signal dependencies, seasonal patterns, and gradual degradation that precede catastrophic failures.

Your challenge is to build a **multi-signal time-series anomaly detection system** capable of identifying point anomalies, contextual anomalies, and collective anomalies across correlated sensor streams, while minimizing false positives and providing explainable root-cause insights.

#### Challenge Requirements:

- 1. Multi-Variate Ingestion:** Ingest and process at least 50 correlated time-series signals (temperature, pressure, vibration, power consumption, flow rates) at 1-second granularity. Handle missing data, sensor dropout, and clock synchronization issues.
- 2. Anomaly Detection Models:** Implement and compare at least three approaches: (a) Statistical methods (Dynamic Threshold, Seasonal Decomposition), (b) ML-based (Isolation Forest, One-Class SVM on engineered features), and (c) Deep learning (LSTM Autoencoder, Transformer-based models like TranAD). The system should detect point anomalies (sudden spikes), contextual anomalies (normal value at abnormal time), and collective anomalies (unusual patterns across multiple signals simultaneously).
- 3. Correlation-Aware Detection:** Go beyond univariate detection by modeling inter-signal correlations. For example, if temperature rises while cooling flow rate drops, this correlated deviation should be flagged even if neither signal individually exceeds its threshold.
- 4. Explainable Alerts:** Each detected anomaly must include: a severity score (critical/warning/info), the contributing signals ranked by importance (using SHAP values or attention weights), a historical comparison showing similar past anomaly patterns, and a suggested investigation path.
- 5. Adaptive Learning:** The system must adapt to concept drift as equipment ages or operating conditions change. Implement an online learning mechanism or periodic retraining pipeline that incorporates operator feedback (confirmed anomaly vs. false positive) to improve detection accuracy over time.

#### Suggested Datasets:

NASA Turbofan Engine Degradation dataset (CMAPSS), Server Machine Dataset (SMD) from OmniAnomaly, Numenta Anomaly Benchmark (NAB), and a custom synthetic multi-signal industrial dataset with labeled anomalies provided by the organizers.

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Technology Requirements:**

Python (PyTorch/TensorFlow, scikit-learn, tslearn), a time-series database (InfluxDB, TimescaleDB, or QuestDB), Apache Kafka for streaming ingestion, and a monitoring dashboard (Grafana or custom Streamlit/Dash application).

**Evaluation Criteria:**

F1-score on labeled anomaly test sets, false positive rate, detection latency (time from anomaly onset to alert), explainability quality (judged by reviewers), and adaptability to concept drift scenarios.

**Domain:**

Time-Series Analytics & Anomaly Detection





## Problem Statement 5:

### Predictive Demand Forecasting for Supply Chain Resilience Using Multi-Source Time-Series Data

#### Description:

Global supply chains face unprecedented volatility from geopolitical disruptions, climate events, demand shocks, and shifting consumer behavior. Traditional demand forecasting approaches, often based on simple moving averages or single-source historical sales data, consistently fail to capture the complex, multi-factorial nature of modern demand patterns. This leads to either excess inventory (tying up capital and risking obsolescence) or stockouts (lost sales and damaged customer relationships).

Your challenge is to build a **multi-source demand forecasting system** that fuses internal sales data with external signals (weather, social media trends, economic indicators, and logistics disruptions) to produce accurate, granular forecasts at the SKU-location-week level, along with probabilistic confidence intervals.

#### Challenge Requirements:

- 1. Multi-Source Data Fusion:** Integrate at least four distinct data sources: historical POS/sales data (2+ years, weekly granularity), weather forecasts and historical weather data (temperature, precipitation, severe weather events), social media trend indices and Google Trends data for relevant product categories, and macroeconomic indicators (consumer confidence index, unemployment rate, fuel prices). Build a feature engineering pipeline that aligns these heterogeneous time-series to a common temporal resolution.
- 2. Hierarchical Forecasting:** Implement a hierarchical forecasting approach that produces coherent forecasts at multiple aggregation levels: national, regional, store-level, and SKU-level. Ensure that lower-level forecasts sum up consistently to higher-level totals using reconciliation methods (MinT, ERM, or bottom-up).
- 3. Model Ensemble:** Build and compare multiple forecasting models: a classical statistical model (ARIMA/ETS/Prophet), a gradient-boosted model (LightGBM/XGBoost with time-series features), and a deep learning model (N-BEATS, TFT, or PatchTST). Combine them using an intelligent ensemble that weights models based on recent backtesting performance.
- 4. Probabilistic Forecasting:** Go beyond point forecasts to provide probabilistic predictions (10th, 50th, 90th percentiles). These uncertainty intervals should drive safety stock calculations, enabling inventory planners to balance service levels against carrying costs.
- 5. Disruption Scenario Simulation:** Build a simulation module that allows planners to model what-if scenarios: What happens to demand if a port is shut down for 2 weeks? How does a 15% price increase affect forecasts? What if a competitor launches a promotional campaign? The system should show forecast adjustments and inventory implications for each scenario.

#### Suggested Datasets:

Favorita Grocery Sales dataset (Kaggle), M5 Forecasting Competition dataset, OpenWeatherMap historical data, Google Trends export files, and synthetic macroeconomic indicator time-series. All data will be provided in CSV/Parquet format.

#### Technology Requirements:

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)



Python (statsmodels, Prophet, LightGBM, PyTorch for deep learning models, hierarchicalforecast library), a data warehouse (DuckDB or PostgreSQL), an orchestration tool (Airflow or Prefect), and a dashboard (Streamlit or Plotly Dash).

**Evaluation Criteria:**

Weighted Mean Absolute Percentage Error (WMAPE) across SKU-location combinations, probabilistic calibration (coverage of prediction intervals), hierarchical coherence score, scenario simulation plausibility, and end-to-end pipeline reliability.

**Domain:**

Time-Series Forecasting & Supply Chain



## Problem Statement 6:

### Agentic MCP Gateway: Building an AI Agent That Orchestrates Third-Party APIs via Model Context Protocol

#### Description:

The Model Context Protocol (MCP) is an emerging open standard that enables AI models to interact with external tools, data sources, and APIs through a unified interface. While individual MCP server implementations exist for popular services (GitHub, Slack, databases, etc.), there is no unified agentic layer that can intelligently orchestrate multiple MCP connections to complete complex, multi-step business workflows autonomously.

Your challenge is to build an **Agentic MCP Gateway**: an AI-powered orchestration layer that connects to multiple third-party services via MCP servers, understands natural language workflow descriptions, decomposes them into executable steps, and orchestrates cross-service operations while handling errors, retries, and human-in-the-loop approvals.

#### Challenge Requirements:

- 1. MCP Server Integration:** Build or configure MCP server connectors for at least four different third-party services spanning different categories: a project management tool (e.g., Jira, Linear, or Trello API), a communication platform (e.g., Slack or Discord API), a data/analytics service (e.g., Google Sheets, Airtable, or a SQL database), and a cloud/DevOps tool (e.g., GitHub, AWS, or monitoring APIs). Each connector must implement the MCP specification for tool discovery, invocation, and response handling.
- 2. Workflow Decomposition Engine:** Build an LLM-powered planning agent that takes a natural language workflow description (e.g., 'When a critical bug is filed in Jira, create a GitHub branch, notify the on-call engineer in Slack, and update the incident tracker spreadsheet') and decomposes it into a directed acyclic graph (DAG) of MCP tool calls with proper data flow between steps.
- 3. Execution Runtime:** Implement a reliable execution runtime that: runs DAG steps in the correct dependency order (with parallelism where possible), handles transient API failures with configurable retry policies, supports human-in-the-loop approval gates for sensitive operations (e.g., deploying code, sending external communications), and maintains an execution log with full observability.
- 4. Context Management:** Implement efficient context management so the agent can: maintain conversation state across multi-turn interactions, pass output data from one MCP tool call as input to subsequent calls, and handle large payloads (e.g., file attachments, long API responses) through summarization or chunking.
- 5. Security and Access Control:** Implement a permission model where: each MCP connection has scoped credentials, the agent cannot escalate its own permissions, sensitive operations require explicit user confirmation, and all API calls are logged for audit purposes.

#### Suggested Datasets:

Mock API servers for each integration category (provided as Docker containers), sample workflow definitions in natural language, test scenarios with expected outcomes, and an MCP specification reference implementation.

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Technology Requirements:**

Python or TypeScript for the agent runtime, an LLM API (Claude, GPT, or open-source) for planning, MCP SDK (@modelcontextprotocol/sdk or equivalent), Docker for mock services, and a web-based UI for workflow definition and monitoring.

**Evaluation Criteria:**

Number of successfully integrated MCP services, workflow decomposition accuracy (correct DAG generation), end-to-end execution success rate on test scenarios, error handling robustness, latency overhead per orchestration step, and security audit compliance.

**Domain:**

[MCP Integrations & Agentic AI](#)



## Problem Statement 6:

### **Agentic MCP Gateway: Building an AI Agent That Orchestrates Third-Party APIs via Model Context Protocol**

#### **Description:**

The Model Context Protocol (MCP) is an emerging open standard that enables AI models to interact with external tools, data sources, and APIs through a unified interface. While individual MCP server implementations exist for popular services (GitHub, Slack, databases, etc.), there is no unified agentic layer that can intelligently orchestrate multiple MCP connections to complete complex, multi-step business workflows autonomously.

Your challenge is to build an **Agentic MCP Gateway**: an AI-powered orchestration layer that connects to multiple third-party services via MCP servers, understands natural language workflow descriptions, decomposes them into executable steps, and orchestrates cross-service operations while handling errors, retries, and human-in-the-loop approvals.

#### **Challenge Requirements:**

- 1. MCP Server Integration:** Build or configure MCP server connectors for at least four different third-party services spanning different categories: a project management tool (e.g., Jira, Linear, or Trello API), a communication platform (e.g., Slack or Discord API), a data/analytics service (e.g., Google Sheets, Airtable, or a SQL database), and a cloud/DevOps tool (e.g., GitHub, AWS, or monitoring APIs). Each connector must implement the MCP specification for tool discovery, invocation, and response handling.
- 2. Workflow Decomposition Engine:** Build an LLM-powered planning agent that takes a natural language workflow description (e.g., 'When a critical bug is filed in Jira, create a GitHub branch, notify the on-call engineer in Slack, and update the incident tracker spreadsheet') and decomposes it into a directed acyclic graph (DAG) of MCP tool calls with proper data flow between steps.
- 3. Execution Runtime:** Implement a reliable execution runtime that: runs DAG steps in the correct dependency order (with parallelism where possible), handles transient API failures with configurable retry policies, supports human-in-the-loop approval gates for sensitive operations (e.g., deploying code, sending external communications), and maintains an execution log with full observability.
- 4. Context Management:** Implement efficient context management so the agent can: maintain conversation state across multi-turn interactions, pass output data from one MCP tool call as input to subsequent calls, and handle large payloads (e.g., file attachments, long API responses) through summarization or chunking.
- 5. Security and Access Control:** Implement a permission model where: each MCP connection has scoped credentials, the agent cannot escalate its own permissions, sensitive operations require explicit user confirmation, and all API calls are logged for audit purposes.

#### **Suggested Datasets:**

Mock API servers for each integration category (provided as Docker containers), sample workflow definitions in natural language, test scenarios with expected outcomes, and an MCP specification reference implementation.

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Technology Requirements:**

Python or TypeScript for the agent runtime, an LLM API (Claude, GPT, or open-source) for planning, MCP SDK (@modelcontextprotocol/sdk or equivalent), Docker for mock services, and a web-based UI for workflow definition and monitoring.

**Evaluation Criteria:**

Number of successfully integrated MCP services, workflow decomposition accuracy (correct DAG generation), end-to-end execution success rate on test scenarios, error handling robustness, latency overhead per orchestration step, and security audit compliance.

**Domain:**

[MCP Integrations & Agentic AI](#)



## Problem Statement 8:

### **GeoSpatial Site Readiness Analyzer: AI-Powered Location Intelligence for Commercial Real Estate and Infrastructure**

#### **Description:**

Selecting optimal locations for new retail stores, warehouses, EV charging stations, telecom towers, or renewable energy installations requires analyzing dozens of geospatial factors: population density, road network accessibility, competitor proximity, land use regulations, environmental risks, and utility infrastructure availability. Currently, this analysis is performed manually by teams of analysts using disconnected GIS tools, spreadsheets, and gut instinct, leading to months-long evaluation cycles and suboptimal site selections.

Your challenge is to build an **AI-powered GeoSpatial Site Readiness Analyzer** that ingests multi-layer geospatial data, computes a composite site readiness score for any given location, and provides interactive map-based visualizations with explainable scoring breakdowns.

#### **Challenge Requirements:**

- 1. Multi-Layer Geospatial Data Ingestion:** Ingest and process at least five distinct geospatial data layers: demographic data (population density, income levels, age distribution from census data), transportation network (road density, proximity to highways/transit, drive-time isochrones), points of interest (competitor locations, complementary businesses, anchor tenants), land use and zoning (commercial/residential/industrial zones, building footprints), and environmental/risk data (flood zones, earthquake risk, air quality indices). Support standard geospatial formats (GeoJSON, Shapefiles, GeoTIFF, and WKT).
- 2. Site Readiness Scoring Model:** Build a configurable scoring model that computes a composite Site Readiness Score (0-100) for any latitude/longitude coordinate. The model should support: configurable weights for each data layer (different use cases prioritize different factors), distance-decay functions (closer proximity to a highway matters more than distant proximity), competitive density analysis (too many competitors in a radius is negative, some is positive as an indicator of market viability), and threshold constraints (hard requirements like minimum population within 5km).
- 3. Spatial Clustering and Hot-Spot Detection:** Implement spatial analysis algorithms (DBSCAN, Getis-Ord Gi\*, or H3 hexagonal binning) to identify clusters of high-potential sites and underserved areas. Visualize hot-spots and cold-spots on the map to guide exploration.
- 4. Interactive Map Interface:** Build an interactive web-based map (using Mapbox GL, Leaflet, Deck.gl, or Kepler.gl) that allows users to: click any point to see its readiness score breakdown, toggle data layers on/off, draw custom polygons to define search areas, compare multiple candidate sites side by side, and export reports for selected locations.
- 5. Route and Accessibility Analysis:** Integrate routing APIs (OSRM, Valhalla, or Google Maps) to compute drive-time and walk-time isochrones from candidate sites. Show catchment area analysis (how many people can reach the site within 10/20/30 minutes by car or transit).

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)

**Suggested Datasets:**

OpenStreetMap data extracts (GeoJSON), census demographic data (GeoJSON with population attributes), FEMA flood zone boundaries, EPA air quality data (point measurements), and a synthetic competitor locations dataset. All spatial data will cover a defined metro area for consistency.

**Technology Requirements:**

Python (GeoPandas, Shapely, H3, scikit-learn for spatial ML), PostGIS for spatial queries, a tile server or vector tile rendering (tippecanoe + MapLibre or Mapbox GL JS), a backend API (FastAPI), and a React frontend with map library integration.

**Evaluation Criteria:**

Number of geospatial layers successfully integrated, scoring model configurability and accuracy (validated against expert-labeled test sites), spatial analysis quality, map interface usability and performance, and accessibility analysis accuracy.

**Domain:**

[Geo Spatial Analytics & Site Detection](#)





## Problem Statement 9:

### Satellite Imagery Change Detection for Urban Expansion Monitoring and Land Use Compliance

#### Description:

Rapid urbanization across the developing world is outpacing the ability of municipal authorities to monitor land use changes, unauthorized construction, encroachments on protected areas, and compliance with zoning regulations. Manual satellite imagery review is expensive, slow, and cannot scale to cover entire metropolitan regions. Meanwhile, freely available satellite imagery (Sentinel-2, Landsat) is growing in resolution and frequency, creating an opportunity for automated change detection.

Your challenge is to build a **satellite imagery change detection system** that automatically identifies, classifies, and quantifies land use changes between two or more time periods, and flags potential compliance violations for review by urban planning authorities.

#### Challenge Requirements:

- 1. Satellite Image Processing Pipeline:** Build an automated pipeline that: downloads satellite imagery for a defined region from public sources (Sentinel-2 via Copernicus or Landsat via USGS EarthExplorer), performs atmospheric correction and cloud masking, creates cloud-free composite images for defined time periods, and aligns multi-temporal images for pixel-level comparison.
- 2. Change Detection Model:** Implement a deep learning-based change detection model (such as Siamese U-Net, BiT, or ChangeFormer) that compares image pairs from different time periods and produces: a binary change mask (changed vs. unchanged pixels), a change classification map (vegetation loss, new construction, water body change, road expansion, etc.), and a confidence score per detected change region.
- 3. Spectral Index Analysis:** Complement the deep learning approach with spectral index computation: NDVI (Normalized Difference Vegetation Index) for vegetation monitoring, NDBI (Normalized Difference Built-up Index) for construction detection, and MNDWI (Modified Normalized Difference Water Index) for water body changes. Show how these indices correlate with and validate the deep learning predictions.
- 4. Compliance Rule Engine:** Implement a configurable rule engine where urban planners can define compliance rules: 'No new construction within 500m of a water body', 'Green cover in residential zones must not decrease below 30%', 'Industrial activity is prohibited in Zone A'. The system should automatically flag detected changes that violate these rules.
- 5. Reporting and Visualization:** Generate automated compliance reports for each detected change area, including before/after imagery, change classification, area measurements (in hectares), affected zone information, and violated rules. Visualize results on an interactive map with time-slider functionality.

#### Suggested Datasets:

Sentinel-2 L2A imagery (10m resolution) for a selected metro area covering 4+ time periods, OSCD (Onera Satellite Change Detection) dataset for model training, land use zoning boundary shapefiles, and labeled change detection ground truth for evaluation.

#### Technology Requirements:

**Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)



Python (rasterio, GDAL, PyTorch/TensorFlow for change detection models), a geospatial database (PostGIS), cloud storage for imagery (S3 or local), a web mapping frontend (Leaflet or Mapbox GL with time-slider), and a reporting module (PDF/HTML generation).

**Evaluation Criteria:**

Change detection F1-score on labeled test imagery, classification accuracy for change types, compliance rule matching precision, processing pipeline throughput (images per hour), and report quality.

**Domain:**

[Geo Spatial Analytics & Remote Sensing](#)



## Problem Statement 10:

### Multi-Agent AI System for Intelligent Resume Parsing, Skill-Set Matching, and API-Ready Talent Intelligence

#### Description:

Recruitment and talent acquisition teams process thousands of resumes daily across diverse formats (PDF, DOCX, LinkedIn exports, plain text), each with inconsistent layouts, terminology, and skill representations. A candidate might list 'React.js' while a job description requires 'ReactJS' or 'React'; one resume uses 'ML Engineering' while the skill database categorizes it as 'Machine Learning Engineering.' Current ATS (Applicant Tracking Systems) rely on rigid keyword matching that misses qualified candidates and surfaces irrelevant ones, leading to poor hiring outcomes and wasted recruiter time.

Your challenge is to build a **Multi-Agent AI system** that intelligently parses resumes from multiple formats, extracts and normalizes skills against a structured skill-set taxonomy, performs semantic matching against job descriptions and role requirements, and exposes the entire pipeline through well-documented REST APIs for seamless third-party integration by HR platforms, job boards, and enterprise systems.

#### Challenge Requirements:

- 1. Multi-Format Resume Parsing Agent:** Build a specialized parsing agent that extracts structured information from resumes in PDF, DOCX, and plain text formats. The agent must handle diverse resume layouts (single-column, multi-column, creative designs, tabular formats) and extract: personal information (name, contact, location), work experience (company, role, duration, responsibilities), education (institution, degree, field, year), skills (technical and soft), certifications, projects, and publications. Use a combination of layout analysis (for PDFs), NLP-based section detection, and LLM-powered extraction to handle edge cases and ambiguous formatting.
- 2. Skill Taxonomy and Normalization Agent:** Build a skill normalization agent that maintains a hierarchical skill-set database (e.g., 'Python' -> 'Programming Languages' -> 'Technical Skills') and maps extracted resume skills to canonical entries. The agent must handle: synonyms and abbreviations ('JS' = 'JavaScript', 'K8s' = 'Kubernetes'), skill hierarchy inference (someone with 'TensorFlow' and 'PyTorch' experience implies 'Deep Learning' proficiency), proficiency level estimation based on context (5 years of Python experience vs. 'familiar with Python'), and emerging skill detection (flagging skills not yet in the taxonomy for human review and addition).
- 3. Semantic Skill-to-Job Matching Agent:** Build a matching agent that goes beyond keyword matching to perform semantic similarity between candidate skill profiles and job requirements. Implement: vector embedding-based skill matching (using sentence transformers or domain-specific embeddings), weighted scoring that considers skill importance (required vs. nice-to-have), experience depth weighting (years of experience, project complexity), gap analysis that identifies missing skills and suggests upskilling paths, and a configurable matching threshold that balances precision vs. recall based on hiring manager preferences.
- 4. Multi-Agent Orchestration Layer:** Design and implement an orchestration framework where the parsing, normalization, and matching agents collaborate through a shared message protocol. The orchestrator should: manage agent lifecycle and task delegation, handle concurrent resume processing (batch mode for bulk uploads), implement retry logic and graceful degradation (if one agent fails, partial results are still returned), and provide observability with per-agent execution

## **Prama Innovations India Pvt. Ltd.**

602 Shapath-5 Building  
SG Highway, Ahmedabad 380015  
[www.prama.ai](http://www.prama.ai)



traces, latency metrics, and quality scores. Use a framework such as LangGraph, CrewAI, AutoGen, or a custom orchestrator.

**5. API Layer for Third-Party Consumption:** Expose the entire pipeline through a production-grade REST API (OpenAPI/Swagger documented) with endpoints for: single resume parsing (POST /api/v1/parse with file upload), batch resume processing (POST /api/v1/parse/batch with async job tracking), skill profile retrieval (GET /api/v1/candidates/{id}/skills), job matching (POST /api/v1/match with candidate ID and job description), skill taxonomy browsing and search (GET /api/v1/skills/taxonomy), and webhook callbacks for async processing completion. Implement API key authentication, rate limiting, request validation, and comprehensive error responses. Provide SDKs or code examples in Python and JavaScript for easy integration.

### **Suggested Datasets:**

A synthetic resume dataset (500+ resumes across 10+ industries in PDF and DOCX formats with varying layouts), a structured skill taxonomy database (5,000+ skills organized hierarchically across technology, business, and domain categories), sample job descriptions (100+ across engineering, data science, product management, and design roles), and a ground-truth matching dataset for evaluation (expert-labeled candidate-job compatibility scores).

### **Technology Requirements:**

Python (LangChain/LangGraph or CrewAI for multi-agent orchestration, sentence-transformers for embeddings, pdfplumber/PyMuPDF for PDF parsing, python-docx for DOCX parsing), a vector database (ChromaDB, Pinecone, or Weaviate) for skill embeddings, PostgreSQL for structured data, FastAPI for the API layer with OpenAPI documentation, Redis for job queuing and caching, and Docker/Docker Compose for deployment.

### **Evaluation Criteria:**

Resume parsing accuracy (field-level F1-score on ground-truth labeled resumes), skill normalization precision (correct canonical mapping rate), matching quality (NDCG and correlation with expert rankings), API completeness (endpoint coverage, documentation quality, error handling), multi-agent orchestration reliability (success rate under concurrent load), and end-to-end latency (single resume processing time < 10 seconds).

### **Domain:**

Multi-Agent AI & Talent Intelligence